eXpressSFU: Rumo à Videoconferência Super-Escalável com SmartNICs

Tuan Tran e S. M. H. Hosseini (University of Colorado Boulder) · Seyeon Kim (Korea University) · Kyunghan Lee (Seoul National University) · Nam Bui (University of Colorado Denver) · Dirk Grunwald e Sangtae Ha (University of Colorado Boulder)

NSDI '26 (23º USENIX Symposium on Networked Systems Design and Implementation), 4–6 de maio de 2026, Renton, WA, EUA. https://www.usenix.org/conference/nsdi26/presentation/tran

Tradução não oficial para o português (pt-BR) de *eXpressSFU: Toward Super-Scalable Video Conferencing with SmartNICs*. Termos técnicos consagrados (como *bitrate*, *throughput*, *jitter*) foram mantidos em inglês quando soam mais naturais assim. Figuras e imagens foram omitidas (permanecem apenas as legendas traduzidas); tabelas e equações foram reproduzidas. A lista de referências é mantida no idioma original. Em caso de dúvida, vale o texto original.

RESUMO

A videoconferência surgiu como uma aplicação crítica da Internet. Diferentemente dos serviços de vídeo sob demanda, a videoconferência de alta qualidade exige latência mínima, pois os fluxos de mídia são gerados, transmitidos, processados, encaminhados e recebidos em tempo real. Nossa análise empírica revela que a latência de processamento no servidor de mídia, em particular nas unidades de encaminhamento seletivo (SFUs), é a principal barreira à escalabilidade. Notavelmente, operações de criptografia, memória e E/S respondem por aproximadamente 79% da latência de processamento dos pacotes de mídia. Neste artigo, apresentamos o eXpressSFU, um sistema de videoconferência re-arquitetado para aumentar significativamente a escalabilidade de videoconferências de grande porte e alta qualidade. Ao desacoplar os planos de controle rápido e de dados do plano de controle lento, o eXpressSFU acelera o pipeline de processamento de pacotes de mídia por meio de tecnologias de rede emergentes, como as placas de rede inteligentes (SmartNICs, *Smart Network Interface Cards*). Os resultados experimentais mostram que nosso sistema reduz a latência de processamento de pacotes por um fator de 8. Essa melhoria permite suportar 3× mais usuários simultâneos, ao mesmo tempo em que reduz o consumo de energia computacional em até 60%.

1 INTRODUÇÃO

A videoconferência teve um crescimento vertiginoso, passando de uma ferramenta empresarial ocasional a um canal de comunicação essencial. Essa mudança foi impulsionada pela crescente demanda por trabalho remoto [29], aulas on-line [24] e colaboração remota [56]. De acordo com o relatório mais recente da Zoom, a plataforma de videoconferência mais popular, há 300 milhões de participantes diários em reuniões [53]. Isso representa um crescimento notável, com taxa média de aumento anual de 87,8 milhões desde o recorde de 10 milhões em dezembro de 2019 [14]. O mercado de videoconferência deve atingir US$ 9,6 bilhões em 2028, crescendo a uma taxa anual composta de 8,35% [8].

As demandas atuais de videoconferência mudaram significativamente em direção a fluxos de vídeo de maior qualidade, como 1080p. A ascensão de aplicações imersivas como realidade aumentada (AR) e realidade virtual (VR), que exigem fluxos de vídeo de qualidade excepcionalmente alta e latência mínima, amplifica ainda mais esse requisito [16, 52]. Para atender a essas demandas crescentes, os sistemas modernos de videoconferência migraram da arquitetura de unidade de controle multiponto (MCU), intensiva em computação, para o modelo mais escalável de unidade de encaminhamento seletivo (SFU) [1, 26]. Ao distribuir as tarefas de codificação e decodificação para os endpoints, as SFUs aumentam a escalabilidade. No entanto, à medida que o número de participantes cresce, a latência de processamento se torna um gargalo crítico, reduzindo a qualidade do vídeo por meio de taxas de bits menores e mais quadros descartados [2, 32].

Pesquisas anteriores tentaram tratar as limitações das SFUs por meio de otimização de software de codecs de vídeo e de pilhas de aplicação [15, 22, 44, 46]. Por outro lado, a abordagem padrão da indústria para lidar com demandas crescentes é a escala horizontal: implantar mais instâncias de servidor para distribuir a carga das conferências [4, 13, 34]. Entretanto, essas abordagens apenas mitigam sintomas, em vez de tratar as restrições arquiteturais fundamentais.

A principal razão pela qual essas abordagens foram ineficazes são os gargalos de latência inerentes à arquitetura atual, que executa toda a pilha da SFU em CPUs x86. Esses gargalos surgem na interseção de duas tendências opostas: o crescimento exponencial das taxas de dados de vídeo e do número de usuários simultâneos [5, 18], frente ao avanço estagnado das tecnologias de CPU e de memória. Essa lacuna crescente exige soluções arquiteturais inovadoras que vão além das otimizações tradicionais de software. Uma abordagem promissora é re-arquitetar o sistema de SFU descarregando (*offloading*) tarefas de processamento de pacotes sensíveis à latência para aceleradores de hardware emergentes, como as SmartNICs.

Com esse objetivo, propomos o eXpressSFU, uma nova arquitetura de SFU fundamentada em evidências empíricas e em uma análise aprofundada do Jitsi Meet [11], uma das plataformas SFU de código aberto mais amplamente adotadas. Ao processar os pacotes de mídia diretamente na SmartNIC, o eXpressSFU pode transmitir o tráfego de saída o mais cedo possível, contornando totalmente o host. O mais importante é que ele acelera significativamente as operações do plano de dados para atender aos rigorosos requisitos de baixa latência da videoconferência de alta qualidade. Como mostra a Figura 1, as soluções convencionais baseadas em software têm dificuldade em lidar com várias conferências simultâneas de 15 usuários a 1080p@24fps, mesmo quando núcleos de CPU adicionais são alocados, o que limita a escalabilidade. Em contraste, o eXpressSFU alcança escalabilidade muito melhor, suportando um número crescente de conferências à medida que mais núcleos ARM da SmartNIC, embora significativamente menos potentes, são utilizados. O eXpressSFU aproveita décadas de esforço de engenharia de software ao reter a maior parte do sistema SFU existente, desacoplando seletivamente tanto o plano de dados quanto o plano de controle

rápido da pilha original da SFU. Essa separação arquitetural permite que operações sensíveis à latência sejam descarregadas e aceleradas na SmartNIC. Além disso, o eXpressSFU mantém ampla compatibilidade com diversos dispositivos de endpoint ao suportar, nativamente na SmartNIC, a troca dinâmica de qualidade tanto para Simulcast quanto para codificação de vídeo escalável (SVC).

Figura 1: Número de conferências simultâneas sem degradação de qualidade. A solução baseada em software tem dificuldade em escalar com os núcleos de CPU do host, enquanto o nosso eXpressSFU mostra escalabilidade significativamente melhor ao tratar fluxos FullHD em núcleos ARM da SmartNIC, com um único núcleo de CPU do host como plano de controle.
(imagem omitida nesta tradução; consulte o artigo original)

Avaliamos o eXpressSFU em um cluster composto por uma workstation equipada com uma SmartNIC Nvidia BlueField-2 [39] e uma CPU Intel i9-14900K servindo como servidor SFU, junto com 100 máquinas bare-metal clientes executando o Chrome Driver, para uma avaliação altamente realista. Medindo o número máximo de usuários ou de conferências simultâneas atendidos pelo eXpressSFU em comparação com um sistema SFU de ponta (por exemplo, o Jitsi Meet), demonstramos as vantagens de escalabilidade do nosso projeto.

As principais contribuições deste artigo são quatro:

- Investigamos as causas subjacentes dos gargalos de escalabilidade por meio de testes de estresse no Jitsi Meet e no Janus com diversas cargas de trabalho. Em particular, fornecemos um detalhamento da latência, identificamos as causas-raiz dos aumentos de latência e explicamos seus impactos sobre a qualidade da mídia.
- Propomos uma nova arquitetura de SFU que descarrega o processamento sensível à latência para uma SmartNIC, preservando a lógica central do sistema para minimizar a disrupção.
- Habilitamos suporte nativo a Simulcast e a SVC na SmartNIC, garantindo compatibilidade com uma ampla gama de dispositivos de endpoint.
- Implementamos o eXpressSFU usando DPDK na SmartNIC, de forma transparente e minimamente invasiva, exigindo apenas modificações mínimas no Jitsi Meet. Avaliamos o eXpressSFU contra a versão mais recente do Jitsi Meet, demonstrando ganhos substanciais de desempenho e de escalabilidade em relação a essa plataforma SFU amplamente utilizada.

2 FUNDAMENTAÇÃO

2.1 Unidade de encaminhamento seletivo

Uma unidade de encaminhamento seletivo (SFU) é uma ponte de pacotes de mídia composta por planos de controle e de dados fortemente acoplados. O plano de controle cuida da adaptação de taxa, coletando o feedback dos receptores para ajustar as taxas de envio e otimizar a qualidade de experiência (QoE), enquanto o plano de dados executa operações em nível de pacote nos fluxos de entrada e de saída, sem transcodificação. Diferentemente da antiga MCU, que decodificava, mixava e recodificava os quadros em um fluxo composto, a SFU encaminha os pacotes diretamente, para obter menor latência e maior escalabilidade.

① **Receber e descriptografar os fluxos (plano de dados).** A SFU recebe fluxos de mídia de qualidades variadas — seja múltiplos fluxos em simulcast ou um único fluxo em camadas em SVC. Cada fluxo, identificado por sua Synchronization Source (SSRC), representa uma opção de taxa de bits/qualidade para atender a diversas condições de rede e de dispositivo. A SFU descriptografa os fluxos de entrada para que o plano de controle possa acessar os metadados para o controle de taxa de bits (§2.2).

② **Estimar e adaptar as taxas de bits (plano de controle).** A SFU analisa os metadados por pacote e o feedback dos receptores para selecionar o nível de qualidade apropriado por endpoint, maximizando o uso da largura de banda e mantendo baixos a latência E2E e o jitter de quadros.

③ **Clonar, recriptografar e encaminhar (plano de dados).** Com base nas decisões do plano de controle, a SFU clona os pacotes selecionados, aplica chaves de criptografia por receptor e os encaminha, garantindo entrega segura e de baixa latência no nível de qualidade escolhido.

2.2 Controle de taxa de bits

Simulcast. O remetente codifica o mesmo vídeo em múltiplos fluxos independentes, com taxas de bits e níveis de qualidade distintos, e os transmite simultaneamente à SFU. A SFU então seleciona e encaminha o fluxo mais adequado a cada receptor, com base em fatores como condições de rede, capacidades do dispositivo e necessidades de exibição. Por exemplo, um usuário com conexão de alta velocidade e uma tela grande pode receber o fluxo de 1080p@24fps, enquanto outro usuário com largura de banda limitada pode receber uma versão de 360p@12fps. Embora o Simulcast ofereça flexibilidade de taxa de bits por usuário, ele impõe demandas significativas de largura de banda de upload e de computação ao remetente [35]. Por isso, vem sendo cada vez mais substituído por uma alternativa mais eficiente: o SVC.

SVC. O remetente codifica o vídeo em um único fluxo composto por múltiplas camadas. Essas camadas consistem em uma camada-base obrigatória — que fornece uma versão de resolução mínima e baixa taxa de quadros — e em camadas de melhoria adicionais construídas sobre ela. Ao codificar todos os níveis de qualidade em um único fluxo, o SVC reduz a redundância, a sobrecarga de codificação e a largura de banda de upload. A SFU encaminha seletivamente as camadas necessárias a cada receptor, garantindo assim a qualidade ótima conforme as restrições individuais de largura de banda e de dispositivo. O mais recente formato de codificação de vídeo do Google, o VP9 [25], oferece suporte nativo a SVC, com escalabilidade temporal e espacial. A temporal ajusta a taxa de quadros, enquanto a espacial ajusta a resolução.

Figura 2: Campos criptografados de um pacote de mídia VP9. A criptografia salto a salto é obrigatória em ambos os modos de criptografia. A criptografia de ponta a ponta é aplicada opcionalmente ao payload de mídia antes da criptografia salto a salto. (a) Modo de criptografia salto a salto: cabeçalhos IP, MAC, UDP e RTP e descritor de payload VP9 em texto claro, mais payload de mídia VP9 e tag de autenticação. (b) Modo de criptografia de ponta a ponta e salto a salto: o payload de mídia VP9 é criptografado de ponta a ponta antes da criptografia salto a salto.
(imagem omitida nesta tradução; consulte o artigo original)

2.3 Criptografia

Criptografia salto a salto (obrigatória). Os sistemas SFU baseados em WebRTC garantem a comunicação segura entre cada usuário e a SFU impondo criptografia salto a salto (*hop-by-hop*),

com o Datagram Transport Layer Security (DTLS) como protocolo de negociação de chaves, o Secure Real-time Transport Protocol (SRTP) para a mídia e o Secure Real-time Transport Control Protocol (SRTCP) para as mensagens de controle. Durante o estabelecimento da conferência, cada participante realiza um handshake DTLS para estabelecer um segredo compartilhado com a SFU. Esse processo cria canais criptografados para a troca de mídia, ao mesmo tempo em que permite que a SFU inspecione os metadados dos pacotes necessários às decisões de encaminhamento. Especificamente, os remetentes criptografam os pacotes antes de enviá-los à SFU. Ao recebê-los, a SFU os descriptografa para extrair informações como os campos de camadas no descritor de payload VP9 (Figura 2a), os recriptografa com chaves por destinatário e os encaminha. Em essência, cada pacote é único, apesar de carregar o mesmo payload de mídia. Embora a criptografia salto a salto proteja os fluxos de mídia em trânsito, ela ainda permite que a SFU acesse o conteúdo da mídia em texto claro, o que levanta possíveis preocupações de privacidade.

Criptografia de ponta a ponta (opcional). Devido às preocupações de privacidade associadas à criptografia salto a salto, a criptografia de ponta a ponta (E2EE) foi introduzida como um recurso avançado. Como ilustra a Figura 2b, o remetente primeiro criptografa o payload de mídia usando chaves conhecidas apenas entre os usuários. Essas chaves E2EE são negociadas entre os usuários no início da conferência, e a SFU atua apenas como retransmissora de mensagens. Depois de aplicada a criptografia de ponta a ponta, a criptografia salto a salto prossegue normalmente, resultando em um payload de mídia duplamente criptografado. Essa configuração impede que a SFU acesse o conteúdo da mídia, mas ainda lhe permite acessar as informações essenciais de camadas.

3 INSIGHTS SOBRE O PROCESSAMENTO DA SFU

3.1 Orçamento de latência para o processamento da SFU

A videoconferência é sensível à latência e ao jitter, isto é, à variância da latência: o aumento da latência cria defasagem entre os participantes, mas o jitter é usado como indicação de que os enlaces estão congestionados e de que o controlador de vídeo deve recuar para uma resolução menor [6]. Se o intervalo entre chegadas de pacotes cair abaixo da taxa de quadros, o controlador de taxa sinaliza ao remetente (ou à SFU) que reduza as taxas de vídeo. Para um fluxo de vídeo a 24 quadros por segundo, isso significa que os quadros precisam chegar em 1/24 s ≈ 41 ms. Quando a SFU replica um fluxo, cada quadro precisa chegar em ≈ 41 ms; como a maioria dos quadros em um fluxo de 1080p@24 fps usa 2 pacotes, o tempo de processamento por pacote deveria ser de 20,5 ms ou menos. Isso é ilustrado na Figura 3, em que dois clientes podem ter RTTs diferentes (20 ms vs. 100 ms), mas cada um precisa receber os dois pacotes que compõem um quadro dentro de 41 ms.

Figura 3: Os receptores ajustam a qualidade do vídeo com base na variância entre quadros, e não na latência total; essa variância entre quadros é determinada pela latência por pacote e pelo número de receptores. (Exemplo: remetente em 1080p@24fps, SFU, RTT = 10 ms, atraso entre quadros = 1/24 s, receptores 1 e 2 em 1080p@24fps.)
(imagem omitida nesta tradução; consulte o artigo original)

Como cada pacote de entrada é duplicado para cada cliente, a latência entre pacotes para um dado cliente depende do número total de clientes atendidos. Se a SFU leva 1 ms para clonar e enviar um único pacote, o tempo para clonar e enviar pacotes a 30 clientes seria de 30 ms, maior que os 20,5 ms permitidos; isso faria os clientes reduzirem a qualidade do vídeo.

Medimos a latência de cada componente do pipeline de processamento de pacotes de mídia. A Figura 4 mostra a latência de processamento por pacote para o Jitsi [11] e o Janus [31], duas SFUs amplamente usadas. Essas latências são difíceis de melhorar devido ao projeto fortemente acoplado dos planos de controle e de dados da arquitetura atual de SFU. Em particular, as operações do plano de dados sensíveis à latência só podem ser executadas depois que o plano de controle tomou uma decisão de encaminhamento por pacote para cada um dos receptores, o que introduz dependências de processamento inevitáveis. Embora o processamento de um pacote seja de apenas ≈ 0,250 ms com 125 usuários, a latência resultante entre pacotes em cada cliente é de 0,25 × 125 = 31,25 ms, acima dos ≈ 20,5 ms que disparam a perda de quadros e a troca para uma qualidade de vídeo reduzida. A Figura 5 mostra a taxa de quadros descartados reportada pelo receptor à medida que aumentamos o número de usuários simultâneos. Como esperado, quando o número de usuários simultâneos chega a 125, ou quando o atraso entre pacotes por usuário excede 20,5 ms, a taxa de quadros descartados aumenta significativamente. No nosso hardware de SFU, o Jitsi Meet só conseguiu atender a um único fluxo com 100 clientes em qualidade 1080p, 180 clientes em 720p e 200 clientes em 360p.

Figura 4: Detalhamento da latência de processamento de vídeo por pacote (Jitsi Meet e Janus; componentes: clonagem de pacote, E/S de pacote, criptografia, descriptografia e controle). À medida que mais usuários entram na conferência, a clonagem de pacotes se torna o gargalo de latência dos sistemas SFU atuais.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 5: O impacto do aumento de usuários simultâneos sobre os quadros descartados e a taxa de bits do vídeo usando uma SFU de um único núcleo. Quando o número de usuários simultâneos chega a 125, a alta latência de processamento de pacotes na SFU leva a descartes substanciais de quadros, disparando feedback dos receptores que força a SFU a reduzir a taxa de envio e, assim, degradando a qualidade do vídeo.
(imagem omitida nesta tradução; consulte o artigo original)

3.2 Oportunidades de aceleração com SmartNIC

Para explorar o potencial da aceleração por SmartNIC, comparamos a latência por operação de tarefas idênticas do plano de dados executadas por uma implementação em software na CPU do host e por nossa contraparte baseada em DPDK nos núcleos ARM da SmartNIC. A Tabela 1 mostra que todas as operações rodam de forma marcadamente mais rápida na SmartNIC do que na CPU x86 quando medidas em um único núcleo em cada caso: a criptografia e a descriptografia aceleram aproximadamente 3× e 8×, respectivamente. A clonagem de pacotes, a principal fonte de latência, melhora cerca de 4×. Notavelmente, a latência de E/S de pacotes é desprezível na SmartNIC.

Operações	CPU do host (ns)	ARM da SmartNIC (ns)
Clonagem de pacotes	1029	241
Descriptografia de pacotes	9732	1253
Criptografia de pacotes	3380	1247
E/S de pacotes	966	24

Tabela 1: Comparação de latência das operações do plano de dados.

Dois fatores impulsionam esses ganhos. Primeiro, a SmartNIC explora a aceleração por processamento em lote (*batch*) do DPDK. Como cada pacote de entrada precisa ser clonado e re-criptografado para vários receptores, a carga de trabalho forma naturalmente lotes consideráveis, o que permite amortizar os custos de criptografia e de clonagem de pacotes. Segundo, o envio e o recebimento de pacotes são tratados inteiramente pelos engines de DMA da NIC, liberando os núcleos ARM da placa. Os pacotes de controle, que respondem por apenas 16% do tráfego total, serão processados no host, já que isso permanece bem dentro das capacidades de processamento da CPU do host. Como demonstrado no Apêndice §A, evitar a travessia do PCIe entre o host e a NIC é crucial para manter a pontualidade dos quadros, o que reforça a vantagem de executar as operações do plano de dados inteiramente na SmartNIC.

4 PROJETO DO EXPRESSSFU

Nesta seção, descrevemos quatro considerações-chave de projeto, seguidas de como realizamos o eXpressSFU, que descarrega as operações do plano de dados para SmartNICs, aumentando drasticamente a escalabilidade. A visão geral do projeto do eXpressSFU é apresentada na Figura 6.

4.1 Considerações de projeto

Aproveitar o desenvolvimento existente de SFUs. É essencial um projeto arquitetural cuidadoso para aproveitar os benefícios de mais de uma década de desenvolvimento de software de SFU, ao mesmo tempo em que se aceleram drasticamente as operações do plano de dados no hardware da SmartNIC. O mais importante é que, como o software de SFU continua evoluindo, um projeto transparente permite que o eXpressSFU adote perfeitamente melhorias futuras.

Separação entre planos de controle e de dados. Uma SFU é um sistema complexo composto por componentes fortemente integrados. Para alcançar transparência e alto desempenho no processamento de pacotes de mídia, é crítico determinar quais componentes devem permanecer na CPU do host e quais devem ser descarregados para a SmartNIC. Embora as SmartNICs ofereçam processamento de pacotes de baixa latência e alta vazão, a CPU x86 do host é muito mais potente e mais adequada para lógicas e algoritmos complexos. Portanto, o eXpressSFU deve equilibrar desempenho e complexidade atribuindo as tarefas à unidade de processamento mais adequada.

Suporte a dispositivos legados e modernos. O SVC tornou-se o padrão de fato na videoconferência por seu uso eficiente de largura de banda e sua troca flexível de camadas de qualidade. No entanto, muitos dispositivos legados ainda não suportam SVC. Para garantir ampla compatibilidade, o eXpressSFU precisa suportar tanto Simulcast quanto SVC, habilitando a funcionalidade de taxa de bits adaptativa independentemente das capacidades do endpoint.

Otimização do sistema SFU para alto desempenho. Maximizar o desempenho do processamento de pacotes nas SmartNICs exige uma distribuição cuidadosa da carga de trabalho no nível dos núcleos. Dois objetivos principais precisam ser atingidos: equilibrar uniformemente a carga entre os núcleos e preservar a ordem dos pacotes por fluxo, já que pacotes fora de ordem podem disparar o controle de congestionamento do WebRTC. Além disso, dadas as diferentes latências das operações, é crucial alocar mais núcleos às tarefas de alta latência e evitar mecanismos de bloqueio ou sincronização. Isso evita travamentos no loop de polling de pacotes, que de outro modo levariam a descartes significativos de pacotes no buffer da NIC.

Figura 6: Visão geral do projeto do eXpressSFU. No host: plano de controle lento (gerenciamento de conferências, estabelecimento de ICE, handshake DTLS, controlador de taxa de bits). Na SmartNIC: plano de controle rápido (aplicação de qualidade de mídia, transformação de fluxos de mídia) e plano de dados (operações criptográficas, E/S de pacotes). Entre eles: APIs de gerenciamento de fluxos, cabeçalhos de pacotes de mídia e decisão de encaminhamento de pacotes, via barramento PCIe.
(imagem omitida nesta tradução; consulte o artigo original)

4.2 Projeto dos planos de controle lento e rápido

Conforme discutido na §3.2, nosso objetivo é descarregar as operações do plano de dados para SmartNICs para atender a requisitos estritos de latência, aproveitando ao máximo o software de SFU existente. Um desafio-chave está na necessidade de o controlador de taxa de bits inspecionar cada pacote de mídia de entrada para a estimativa de taxa. No entanto, nossa observação-chave é que o controlador só precisa dos metadados nos cabeçalhos dos pacotes e do tamanho do pacote para realizar a estimativa — ele não precisa acessar o payload de mídia. Aproveitando esse insight, dividimos o tratamento de pacotes em dois caminhos paralelos. No caminho lento, o cabeçalho do pacote é separado e encaminhado ao controlador de taxa de bits que roda no host para a estimativa de taxa, enquanto o pacote de mídia completo permanece na SmartNIC para processamento de alto desempenho no caminho rápido. A Figura 6 ilustra a arquitetura resultante do eXpressSFU.

Plano de controle lento. As tarefas complexas de gerenciamento que toleram latência maior são tratadas pela CPU do host, aproveitando sua flexibilidade. O plano de controle lento é responsável por tarefas como a negociação de chaves DTLS, o estabelecimento de canais de dados, o gerenciamento de conferências (por exemplo, entrada e saída de participantes) e, mais importante, o controle de taxa de bits. Para integrar ao nosso sistema, fazemos apenas modificações mínimas no controlador de taxa de bits e no gerenciador de conferências existentes — principalmente expondo metadados dos fluxos de mídia e encaminhando decisões de qualidade por meio de nossas APIs de gerenciamento de fluxos — deixando todos os demais módulos da SFU completamente inalterados.

Plano de controle rápido. Devido ao projeto transparente do eXpressSFU, o plano de controle lento seleciona a qualidade de vídeo apropriada para cada receptor, mas não participa diretamente da transmissão dos pacotes de mídia. Em vez disso, a aplicação das decisões de qualidade é delegada ao plano de controle rápido, que roda na SmartNIC e atua como intermediário entre os planos de controle lento e de dados. Ele recebe as decisões de qualidade do controlador de taxa de bits no plano

de controle lento e instrui o plano de dados a duplicar seletivamente os pacotes de acordo, garantindo que as políticas de qualidade sejam aplicadas pacote a pacote.

4.3 Descarregando o plano de dados para a SmartNIC

Plano de dados. Em uma SFU, os pacotes de mídia de entrada precisam ser descriptografados para extrair os metadados de encaminhamento, como os IDs das camadas temporal e espacial. Com base no feedback dos receptores, os pacotes são então clonados seletivamente e criptografados no local (*in-place*) antes de serem encaminhados. Como mostrado na §3, as operações do plano de dados contribuem significativamente para a sobrecarga de processamento das SFUs baseadas em software. No entanto, essas operações podem ser aceleradas de forma dramática em SmartNICs (Tabela 1). Para atender às exigências de latência de videoconferências de grande porte e alta resolução, o eXpressSFU delega as operações do plano de dados à SmartNIC. Como o plano de dados trata diretamente a E/S de pacotes, ele também precisa suportar Simulcast e codificação de vídeo escalável (SVC) para habilitar recursos de taxa de bits adaptativa.

SVC. Cada pacote de mídia corresponde a uma camada espacial e temporal específica. Dada a camada-alvo especificada pelo controlador de taxa de bits, o plano de controle rápido aplica a qualidade determinando se um pacote deve ser entregue a um receptor específico. Se um pacote pertence a uma camada acima da atribuída como alvo, o plano de controle rápido pode instruir o plano de dados a descartar o pacote para aquele receptor. Inversamente, se o pacote está dentro da camada-alvo, ele é encaminhado normalmente. Esse mecanismo garante que cada receptor receba apenas as camadas de vídeo apropriadas à sua largura de banda atual.

Simulcast. No simulcast, cada fluxo de entrada representa um nível de qualidade diferente (por exemplo, 360p, 720p, 1080p). O controlador de taxa de bits, executando no plano de controle lento, seleciona o fluxo mais adequado para cada receptor com base em sua largura de banda disponível. Essa seleção é passada ao plano de controle rápido, que a aplica. Depois que o plano de dados clona os pacotes, o plano de controle rápido reescreve os cabeçalhos RTP dos pacotes de saída para corresponder ao mapeamento de fluxo selecionado, garantindo a entrega da qualidade correta a cada receptor.

4.4 Planos de controle e de dados fracamente acoplados

Nas SFUs convencionais baseadas em software, os planos de controle e de dados são fortemente acoplados, exigindo que todo pacote de mídia passe pela lógica de controle, o que dificulta atender aos requisitos de latência da conferência de alta qualidade. O eXpressSFU quebra essa dependência desacoplando o plano de controle, tolerante a latência, do plano de dados, sensível à latência. Isso permite que o plano de dados seja acelerado por hardware e escalado de forma independente na SmartNIC, enquanto a lógica de controle complexa permanece no host. Para viabilizar a adaptação oportuna da taxa de bits, o eXpressSFU introduz um mecanismo leve, assíncrono e de coordenação em malha fechada. Como mostra a Figura 6, os pacotes de mídia completos permanecem na SmartNIC, enquanto apenas os cabeçalhos dos pacotes e o tráfego de controle são enviados ao plano de controle lento. O controlador de taxa de bits e o módulo DTLS processam essa informação e atualizam o plano de controle rápido, por meio das APIs de gerenciamento de fluxos, com chaves criptográficas, mapeamentos de fluxo e diretivas de camada-alvo. Como os caminhos lento e rápido rodam em paralelo, a coordenação só é disparada mediante eventos relacionados à mídia, como mudanças na largura de banda do usuário.

5 O SISTEMA EXPRESSSFU

Esta seção apresenta o eXpressSFU, um sistema projetado para escalar videoconferências grandes e de alta qualidade descarregando o processamento de pacotes sensível à latência para SmartNICs, mantendo a transparência. A Figura 7 ilustra a visão geral do sistema eXpressSFU. Os pacotes de mídia de entrada chegam primeiro ao Forwarding Execution Engine (§5.3) na SmartNIC, onde o Crypto Engine remove a criptografia salto a salto. Em seguida, os pacotes entram no Path Demultiplexer, que divide o tráfego no caminho lento (controle) e no caminho rápido (dados):

Figura 7: Visão geral do sistema eXpressSFU. Na SmartNIC: Forwarding Execution Engine (Media Crypto Engine, Path Demultiplexer, Packet Replication & Rewriter) e Forwarding Logic Unit (Selective Replication Engine, Stream Mapper). No host: Bitrate Controller e Conference Manager do sistema existente. Caminhos lento e rápido entre remetentes e receptores.
(imagem omitida nesta tradução; consulte o artigo original)

Caminho lento. O demultiplexador encaminha dois tipos de tráfego ao host: (i) pacotes RTCP contendo informações de controle, como o feedback de largura de banda por receptor e os handshakes DTLS, e (ii) o cabeçalho RTP de cada pacote, juntamente com seu descritor de payload VP9, que revela as camadas temporal (TID) e espacial (SID) do pacote. O Bitrate Controller (§5.1) no lado do host estima a taxa de entrada do remetente e usa o feedback de largura de banda dos receptores para tomar decisões de qualidade. Essas decisões — expressas como diretivas de "stream-mapping" e de "target-layers" — são enviadas de volta à SmartNIC, onde a Forwarding Logic Unit (§5.2) as aplica para controlar, no nível de pacote, a qualidade de envio para cada receptor nos fluxos de saída.

Caminho rápido. Os pacotes de mídia completos permanecem na SmartNIC. O Path Demultiplexer encaminha cada pacote ao Packet Replication & Rewriter (PRR), que cria réplicas por receptor. O fato de um receptor dever ou não receber um pacote depende da decisão de qualidade do Bitrate Controller. O Selective Replication Engine (SRE) examina o cabeçalho do pacote e seus campos TID/SID. Usando as diretivas de camada-alvo do Bitrate Controller, o SRE determina exatamente quantas réplicas devem ser geradas para satisfazer o requisito de qualidade de cada receptor. O Stream Mapper então reescreve SSRCs, números de sequência, timestamps e outros campos do cabeçalho RTP de acordo com as diretivas de stream-mapping. Por fim, o Crypto Engine reaplica a criptografia salto a salto, e a SmartNIC transmite os pacotes de mídia diretamente aos receptores, completando o pipeline do plano de dados sem envolver o host.

5.1 Controlador de taxa de bits

Para processar totalmente os pacotes de mídia na SmartNIC, o controlador de taxa de bits que roda no host precisa fornecer à SmartNIC informações essenciais relacionadas aos fluxos. No entanto, essas interações devem permanecer simples e fáceis de integrar em uma ampla gama de plataformas. Para isso, desenvolvemos quatro APIs mínimas, porém suficientes, para

transmitir informações de fluxo e de DTLS à SmartNIC, resumidas na Tabela 2.

Para atender aos requisitos de criptografia salto a salto do WebRTC, a função `send_keys()` transmite à SmartNIC as chaves de criptografia e de salt de cada fluxo de mídia após a negociação DTLS entre a SFU e cada usuário. Como a SmartNIC não tem o conceito de usuários ou de conferências, `send_mapping()` estabelece o mapeamento entre os fluxos de entrada (remetente) e de saída (receptor), incluindo seu deslocamento de número de sequência. Esse mapeamento é especialmente crucial para o Simulcast, pois determina qual fluxo de qualidade deve ser encaminhado com base na largura de banda de cada receptor.

Para o SVC, a SmartNIC precisa conhecer as camadas de qualidade-alvo — especificamente SID para a resolução e TID para a taxa de quadros — atribuídas a cada fluxo de saída. Isso é tratado pela função `send_layers()`. Quando um participante sai, a função `expire_stream()` informa à SmartNIC o ID do fluxo associado (SSRC), permitindo que ela libere recursos e evite estouro de memória. A Figura 8 ilustra como essas funções operam no sistema de conferência.

Método	Descrição
`send_keys(long ssrc, byte[] encKey, byte[] saltKey);`	Envia à SmartNIC as chaves de criptografia e de salt de um fluxo.
`send_mapping(long rx_ssrc, long tx_ssrc, long delta_seqNo);`	Envia à SmartNIC um mapeamento entre fluxos de entrada e de saída.
`send_layers(long tx_ssrc, int TID, int SID);`	Envia à SmartNIC as camadas SVC-alvo de um fluxo de saída.
`expire_stream(long ssrc);`	Envia à SmartNIC a mensagem de expiração de um fluxo.

Tabela 2: APIs de gerenciamento de fluxos de mídia do eXpressSFU, do host para a SmartNIC.

Figura 8: APIs de gerenciamento de conferências no eXpressSFU: ① entrada na conferência (requisição/resposta de entrada e handshake DTLS entre usuários e Conference Manager); ② `send_keys()` e `send_mapping()` ao Crypto Engine e à Forwarding Logic Unit; ③ laço de streaming (cabeçalho RTP e descritor de payload, feedback de largura de banda do receptor, `send_mapping()`/`send_layers()`); ④ saída da conferência e `expire_stream()`.
(imagem omitida nesta tradução; consulte o artigo original)

Conforme ①, quando um participante entra em uma conferência, o gerenciador de conferências primeiro realiza a negociação SDP [27] para trocar parâmetros de mídia e de transporte. Em seguida, estabelece as sessões de mídia usando o protocolo ICE [50]. Se uma conexão direta não puder ser formada, o sistema recorre a STUN ou TURN [37, 51]. Um handshake DTLS [49] então se segue, estabelecendo chaves compartilhadas para a criptografia salto a salto [3].

Após as etapas de configuração da conferência, como mostrado em ②, o gerenciador de conferências invoca as funções `send_keys()` e `send_mapping()` para transmitir à SmartNIC as chaves criptográficas e os mapeamentos iniciais de fluxo.

Na etapa ③, o controlador de taxa de bits seleciona o nível de qualidade apropriado para cada fluxo de saída dentro do laço de streaming. Os pacotes de mídia de entrada são pré-processados e truncados na cauda (*tail-trimmed*) na SmartNIC, retendo apenas o cabeçalho RTP e o descritor de payload VP9 para o controlador de taxa de bits. A CPU do host inspeciona esses cabeçalhos exclusivamente para a estimativa de taxa de bits e depois descarta os pacotes, pois todos os fluxos de saída são gerados de forma independente e inteiramente na SmartNIC. Isso resulta em carga de CPU desprezível e melhora a privacidade, ao manter o conteúdo da mídia confinado à camada de enlace de dados, ao contrário da solução do estado da arte, em que o conteúdo da mídia era exposto na camada de aplicação.

SVC. O controlador de taxa de bits chama `send_layers()` para configurar o SRE da SmartNIC, garantindo que apenas pacotes que correspondam às camadas-alvo sejam replicados e enviados. As camadas-alvo são ajustadas dinamicamente com base nas taxas de bits dos fluxos de entrada e no feedback de largura de banda dos receptores. `send_layers()` é invocada apenas quando uma decisão de qualidade muda. O controlador de taxa de bits também usa `send_mapping()` para atualizar o Stream Mapper da SmartNIC, que reescreve os cabeçalhos RTP dos pacotes de saída.

Simulcast. Cada nível de qualidade é entregue como um fluxo RTP separado, eliminando a necessidade de camadas intrafluxo; assim, `send_layers()` nunca é usada. Em vez disso, o controlador de taxa de bits invoca `send_mapping()` durante as trocas de qualidade, permitindo que o Stream Mapper reescreva os cabeçalhos RTP em tempo real.

Seja com Simulcast ou SVC, a SmartNIC continua encaminhando pacotes de mídia até receber novas configurações de qualidade-alvo, via "target-layers" ou "stream-mapping". Como mostrado em ④, quando um participante sai, o controlador de taxa de bits invoca `expire_stream()` para liberar os recursos associados ao fluxo (por exemplo, operações criptográficas do DPDK, buffers de chaves). Isso também evita o reuso de chaves criptográficas obsoletas caso SSRCs sejam reatribuídos a novos fluxos.

5.2 Forwarding Logic Unit

No WebRTC, os receptores não têm conhecimento do Simulcast nem do SVC; portanto, toda troca de qualidade feita pelo eXpressSFU precisa ser completamente transparente. Isso significa que o receptor deve experimentar um fluxo contínuo, com números de sequência e timestamps crescentes de forma sequencial, independentemente de ocorrer ou não uma troca de qualidade.

A Forwarding Logic Unit (FLU) cuida da troca de qualidade no nível de pacote e consiste em dois submódulos: o SRE e o Stream Mapper. O Stream Mapper suporta Simulcast transformando os cabeçalhos RTP e remapeando os pacotes de mídia de entrada para os fluxos de saída de acordo com as instruções do controlador de taxa de bits. Trabalhando em conjunto com o Stream Mapper, o SRE habilita o SVC ao impor a taxa de bits do fluxo de saída de acordo com a decisão de camada-alvo do controlador de taxa de bits.

Stream Mapper. O fluxo de pacotes do Simulcast é visualizado na Figura 9. Ele é relativamente simples de suportar porque os fluxos de entrada são codificados em níveis de qualidade distintos, cada um identificado por um SSRC exclusivo. Durante uma mudança de qualidade, o controlador de taxa de bits envia um stream mapping atualizado usando `send_mapping()`. Enquanto o identificador do fluxo de saída `tx_ssrc` permanece inalterado, o identificador do fluxo de entrada `rx_ssrc` é atualizado. Para facilitar a troca, aguarda-se um quadro-chave (*keyframe*) do novo fluxo, depois o mapeamento antigo é substituído pelo novo

e os pacotes continuam sendo encaminhados sem interrupção. Isso garante que o decodificador no receptor consiga decodificar corretamente os quadros do fluxo recém-mapeado.

Figura 9: Encaminhamento de pacotes do eXpressSFU. No SVC, ele encaminha apenas as camadas que cada receptor consegue sustentar. No Simulcast, ele escolhe o fluxo mais adequado para cada receptor. (Fluxos de 360p@12fps, 720p@24fps e 1080p@24fps; camadas SID/TID do SVC.)
(imagem omitida nesta tradução; consulte o artigo original)

O processo de reescrita do cabeçalho RTP envolve três etapas. Primeiro, o último número de sequência de saída é incrementado em 1; se não houver histórico, aplica-se o `delta_seqNo` de `send_mapping()` para mapear o pacote de entrada ao primeiro pacote de saída. Segundo, o timestamp é atualizado — copiado do pacote transmitido mais recentemente ou incrementado por Δt_{stamp} se o pacote inicia um novo quadro. Aqui, Δt_{stamp}, o intervalo entre quadros do fluxo de entrada, é aplicado a todos os fluxos de saída associados para manter um ritmo (*pacing*) consistente. Por fim, o SSRC do remetente é substituído pelo do receptor, associando corretamente o pacote ao seu fluxo de saída.

Selective Replication Engine. O fluxo de pacotes do SVC também é visualizado na Figura 9. Suportar SVC é mais complexo porque várias camadas de qualidade são codificadas em um único fluxo, identificado por um único SSRC. O primeiro desafio é determinar o pacote ideal para subir ou descer entre camadas espaciais e temporais. O segundo desafio é decidir se um pacote deve ser encaminhado a cada receptor para alcançar a resolução/taxa de quadros desejada especificada pelo controlador de taxa de bits.

Troca de camada temporal. O processo de troca de camadas temporais é ilustrado no Algoritmo 1. Enquanto reduzir uma camada temporal pode ocorrer a qualquer momento, aumentá-la exige esperar pelo bit U no descritor de payload VP9 (linha 8). O bit U serve como ponto de comutação para cima temporal, indicando se o quadro da camada atual depende de quadros de camadas anteriores da mesma camada temporal [55]. A camada temporal pode ser aumentada em uma ou duas camadas por vez.

Algoritmo 1: Troca de camada temporal no SVC.

Entrada: cur_{TID} // camada temporal em uso, p. ex., 15 fps
tar_{TID} // camada temporal desejada, p. ex., 30 fps
p // pacote de vídeo recebido, com os campos p.TID, p.U_bit
Saída: decisão sobre trocar ou não para a nova camada temporal

```
3  Função TemporalSwitch(cur_TID, tar_TID, p):
4      se tar_TID ≠ cur_TID então
           // reduz a taxa de quadros
5          se tar_TID < cur_TID então
6              cur_TID ← tar_TID
           // aumenta a taxa de quadros
7          senão
               // espera pelo ponto de comutação para cima
8              se p.U_bit e cur_TID < p.TID ≤ tar_TID então
9                  cur_TID ← p.TID
10             senão
                   // não é possível trocar o TID neste pacote
11     senão
           // sem mudança de TID
```

Troca de camada espacial. A troca de camadas espaciais é ilustrada no Algoritmo 2. Reduzir uma camada espacial é possível no início de um novo quadro, conforme indicado pelo bit B (linha 6). Já aumentar exige esperar que o bit P seja 0 (linha 10), sinalizando que o quadro não depende de predição entre imagens (*inter-picture*) [55]. Isso permite subir de (SID−1) para (SID), e apenas uma camada espacial pode ser aumentada por vez (linha 10).

Algoritmo 2: Troca de camada espacial no SVC.

Entrada: cur_{SID} // camada espacial em uso, p. ex., 720p
tar_{SID} // camada espacial desejada, p. ex., 1080p
p // pacote de vídeo recebido, com os campos p.SID, p.B_bit, p.P_bit
Saída: decisão sobre trocar ou não para a nova camada espacial

```
3  Função SpatialSwitch(cur_SID, tar_SID, p):
4      se tar_SID ≠ cur_SID então
           // reduz a resolução
5          se tar_SID < cur_SID então
               // espera por um novo quadro
6              se p.B_bit então
7                  cur_SID ← tar_SID
8              senão
                   // não é possível trocar o SID neste pacote
           // aumenta a resolução; sobe a partir do SID imediatamente inferior e sem predição entre imagens em uso
9          senão
10             se cur_SID + 1 = p.SID e não p.P_bit então
11                 cur_SID ← p.SID
12             senão
                   // não é possível trocar o SID neste pacote
13     senão
           // sem mudança de SID
```

Decisão de encaminhamento de pacotes. O SRE determina quantas cópias de um pacote são necessárias para atender aos receptores pretendidos. Com base no TID, os receptores que exigem taxas de quadros menores não recebem pacotes de camadas temporais mais altas. De forma semelhante, com base no SID, os receptores que visam resoluções menores não recebem pacotes de camadas espaciais mais altas e também podem pular camadas inferiores que não contribuem como referência. Concluída essa filtragem, o SRE instrui o PRR a gerar o número exato de cópias do pacote, evitando duplicação desnecessária.

Sinalização do quadro final. O bit M no cabeçalho RTP indica o último pacote de um quadro. Como as camadas espaciais inferiores contêm menos pacotes, o eXpressSFU precisa reescrever o bit M no cabeçalho RTP para sinalizar adequadamente o fim de um quadro.

5.3 Forwarding Execution Engine

O Forwarding Execution Engine (FEE) é responsável pela manipulação direta dos pacotes de mídia e é onde ocorre a aceleração do plano de dados. Ele consiste em três submódulos: o Media Crypto Engine, o Path Demultiplexer e o PRR.

Media Crypto Engine. Na chegada do pacote, os pacotes de mídia (RTP) são descriptografados e autenticados usando as chaves de criptografia e de salt fornecidas pelo gerenciador de conferências. Os pacotes de controle (RTCP), porém, contornam a SmartNIC e são descriptografados no host. Antes de os pacotes de mídia serem enviados, o Media Crypto Engine reaplica a criptografia salto a salto e anexa uma tag de autenticação ao final do payload de mídia para garantir a integridade do pacote e impedir adulterações. Para acelerar as operações criptográficas, o sistema aproveita as capacidades de processamento em lote do DPDK. Cada núcleo da SmartNIC mantém sua própria sessão criptográfica DPDK dedicada (`rte_crypto_sym_xform`) [21],

para eliminar bloqueios entre núcleos e garantir processamento paralelo e livre de contenção.

Path Demultiplexer. Após a descriptografia, o cabeçalho RTP é analisado, e o campo de tipo de payload é usado para demultiplexar mídia e tráfego que não é de mídia. Os pacotes que não são de mídia são encaminhados imediatamente ao host para tratamento. Os cabeçalhos dos pacotes de mídia e os descritores de payload VP9 também são enviados ao host, habilitando a estimativa da taxa de bits do fluxo de entrada. Enquanto isso, os pacotes de mídia completos são passados ao PRR para processamento no caminho rápido.

Packet Replication & Rewriter. O cabeçalho RTP e o descritor de payload VP9 são processados primeiro pelo SRE, que determina quantas cópias de um pacote são necessárias para atender a todos os receptores pretendidos. O módulo PRR então cria exatamente o número necessário de duplicatas, evitando tráfego de saída desnecessário. O cabeçalho de cada pacote replicado é modificado pelo Stream Mapper, que reescreve o cabeçalho RTP para mapear o pacote de entrada ao pacote de saída apropriado. Além disso, os IPs/portas de origem e de destino são trocados antes de se entregarem os pacotes de saída ao Media Crypto Engine para recriptografia.

Processamento mestre-escravo consciente de latência. O núcleo mestre (*master*) recebe os pacotes de entrada e atribui cada fluxo a um núcleo trabalhador (*worker*) em rodízio (*round-robin*). Cada núcleo trabalhador é responsável por executar as operações do plano de dados do fluxo atribuído e por transmitir o tráfego de saída diretamente de sua própria fila, evitando um gargalo centralizado no núcleo mestre sob alta carga. Quando há muitos receptores, a carga de descriptografia é transferida ao núcleo mestre, que também duplica o fluxo de mídia de entrada. Cada núcleo trabalhador então recebe o mesmo conjunto de pacotes descriptografados e fica responsável por encaminhá-los a um subconjunto disjunto de receptores. Essa abordagem ajuda a reduzir a latência de processamento acumulada para atender aos rígidos requisitos de latência de conferências grandes e de alta qualidade.

6 IMPLEMENTAÇÃO

6.1 Pilha do host

A pilha do host do eXpressSFU é implementada modificando o Jitsi Videobridge [12] para incorporar as funções de gerenciamento de fluxos descritas na Tabela 2. Notavelmente, habilitar o offload para a SmartNIC exige apenas modificações mínimas na base de código original da SFU (254 de 69.409 linhas de código). Os demais componentes do sistema Jitsi Meet permanecem inalterados, incluindo um servidor XMPP [9] e uma aplicação web React [10]. O Jitsi Meet foi escolhido como sistema de referência (*baseline*) porque suporta tanto Simulcast quanto SVC com VP9, o codec de vídeo mais recente do Google. Além disso, o Jitsi Meet tem uma grande base de usuários, o que aumenta o potencial de adoção do eXpressSFU.

As funções de gerenciamento de fluxos se comunicam com a SmartNIC pelo barramento PCIe usando o DOCA Comm Channel da NVIDIA [42]. Esse canal de comunicação é seguro, pois apenas usuários com privilégios de root podem acessá-lo. Embora as mensagens de controle sejam enviadas pela via PCIe por razões de segurança e de baixa latência, outros tipos de pacotes — como pacotes de mídia truncados na cauda ou pacotes de controle RTCP — são transmitidos e recebidos pela interface voltada ao host da SmartNIC.

6.2 Pilha da SmartNIC

A pilha da SmartNIC é implementada na plataforma BlueField-2 [39] como uma aplicação DPDK [47] executando em oito núcleos ARM. O Open vSwitch [48] e as Scalable Functions [43] são configurados para que os pacotes de entrada sejam primeiro direcionados ao programa DPDK para processamento. Para habilitar a criptografia salto a salto, é empregada a biblioteca Cryptodev Scheduler Poll Mode Driver (PMD) [30] do DPDK. Vale notar que não há aceleração criptográfica por hardware utilizável pelo DPDK para AES-GCM-128/256, que é a suíte usada pelo Jitsi Meet para criptografia, descriptografia e autenticação de pacotes.

7 AVALIAÇÃO

Nesta seção, avaliamos o desempenho do eXpressSFU contra uma solução de SFU baseada em software (por exemplo, o Jitsi Meet). Nossa avaliação é guiada pelas seguintes perguntas. **Q1:** Qual é o número máximo de conferências simultâneas adicionais que o eXpressSFU consegue suportar em comparação com a linha de base em software, em tamanhos típicos de conferência? Ver §7.2. **Q2:** Por qual fator o processamento de pacotes de mídia pode ser acelerado na SmartNIC em relação à CPU do host? Ver §7.3. **Q3:** Com que eficácia o eXpressSFU consegue adaptar a taxa de envio em resposta a variações na largura de banda disponível do receptor? Ver §7.4. **Q4:** Qual é a economia de energia esperada para provedores de serviço que adotarem o eXpressSFU? Ver §7.5.

7.1 Configuração experimental

Nossos experimentos são realizados em um cluster composto por 100 máquinas bare-metal como clientes de teste de carga e uma workstation como SFU. Todas as máquinas são conectadas back-to-back por um switch L2 Top-of-Rack, operando a 100 Gbps. A especificação das máquinas cliente é listada na Tabela 3a, enquanto a da máquina eXpressSFU é listada na Tabela 3b.

O eXpressSFU e o Jitsi Meet são implantados em infraestrutura idêntica para garantir uma comparação justa. Ao executar o Jitsi Meet, a SmartNIC é configurada para que os pacotes contornem o núcleo ARM e sejam tratados pela CPU do host. Um framework de orquestração baseado em Ansible também automatiza os processos de implantação, execução e coleta de logs em todas as máquinas cliente. Os cenários de avaliação são apresentados na Tabela 3c, e cada conferência tem um único remetente, sendo os demais receptores. Essa configuração de conferência ajuda a detectar de forma confiável a degradação de qualidade, coletando relatórios de resolução e de atraso entre quadros de cada receptor por meio do método `getStats()` [19].

Componente	(a) Máquina cliente
CPU	Intel E5-2680 v2 @ 2,8 GHz (20 núcleos)
Memória	64 GB DDR3
NIC	10G Mellanox ConnectX-3

Componente	(b) Máquina eXpressSFU
CPU	Intel i9-14900K (24 núcleos no total)
Memória	64 GB DDR5
NIC	100G Nvidia BlueField-2

Tamanho da conferência	# de usuários	(c) Qualidade de vídeo
Pequena	15	720p/1080p@24fps
Média	50	720p/1080p@24fps
Grande	100	720p/1080p@24fps

Tabela 3: Configuração da avaliação. (a) Especificação das máquinas cliente; (b) especificação da máquina eXpressSFU; (c) cenários de avaliação.

7.2 Impacto da escala de usuários e da qualidade da mídia

A Figura 10 apresenta o número de conferências simultâneas suportadas em cenários de grande escala antes que ocorra degradação da qualidade do vídeo, variando o número de núcleos Intel ou ARM dedicados ao processamento de pacotes. Esse cenário é um dos mais desafiadores para escalar sistemas SFU, particularmente no contexto de webinars on-line. A complexidade é notavelmente exacerbada a 1080p@24fps, em que o orçamento de latência é restrito devido à elevada qualidade de mídia e ao número substancial de usuários simultâneos (ver §7.3). Nessa resolução, o Jitsi Meet tem dificuldade de escalar efetivamente mesmo com núcleos de CPU adicionais. Em contraste, o eXpressSFU demonstra escalabilidade linear à medida que mais núcleos ARM são provisionados. A 720p@24fps, o eXpressSFU ainda supera o Jitsi Meet, embora a diferença de escalabilidade seja menor, já que o orçamento de latência é mais flexível. Tendências semelhantes podem ser observadas em conferências de porte médio, tipicamente usadas em aulas on-line, como mostra a Figura 11.

As conferências de porte pequeno — de longe as mais comuns — são ilustradas na Figura 12. Em configuração de alta resolução, o eXpressSFU escala particularmente bem, graças ao seu desempenho de descriptografia de baixa latência, detalhado na Tabela 1. A descriptografia é a operação computacionalmente mais cara do plano de dados e se torna um gargalo significativo de CPU ao lidar com um grande número de fluxos de entrada. Em conferências pequenas e de baixa qualidade, o eXpressSFU e o Jitsi Meet escalam de forma igualmente boa.

Figura 10: Conferências grandes (100 usuários): número de conferências simultâneas para Jitsi Meet e eXpressSFU em 720p e 1080p, conforme o número de núcleos de CPU ou ARM.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 11: Conferências médias (50 usuários).
(imagem omitida nesta tradução; consulte o artigo original)

Figura 12: Conferências pequenas (15 usuários).
(imagem omitida nesta tradução; consulte o artigo original)

O BlueField-2 contém apenas 8 núcleos operando a 2 GHz, enquanto a CPU do host contém 24 núcleos operando a 2,5–6,0 GHz. O eXpressSFU alcança melhor desempenho por causa do nosso projeto focado em particionar o trabalho sensível à latência para o BlueField-2, aliado ao DPDK e a interfaces de rede de baixa latência. Como os pacotes de controle respondem por apenas 16% do tráfego total, o eXpressSFU deve ser capaz de escalar para múltiplas SmartNICs BlueField-2 ou para a placa BlueField-3, mais rápida, que dobra a contagem de núcleos ARM e inclui outras acelerações.

7.3 Melhorias na latência de processamento

A Figura 13 mostra a latência medida de cada componente do pipeline de processamento de pacotes de mídia[1] na SmartNIC, usando a mesma metodologia da §3. Mesmo com 150 usuários, a latência do eXpressSFU é 1/8 da do Jitsi, aumentando o número de usuários que podem ser atendidos antes que a latência dos pacotes cause descarte de quadros. A ausência de sobrecarga do plano de controle decorre do projeto inteligente de desacoplamento do eXpressSFU, que isola a lógica de controle lento dos planos de controle rápido e de dados, sensíveis à latência. A latência de E/S de pacotes é mínima devido ao offload para a SmartNIC; o núcleo ARM apenas emite operações de DMA assíncronas, enquanto o hardware da NIC gerencia de forma independente o envio e o recebimento de pacotes.

Figura 13: Comparação de latência entre Jitsi Meet, Janus e eXpressSFU. O eXpressSFU reduz a latência em 8× com 150 usuários.
(imagem omitida nesta tradução; consulte o artigo original)

Graças ao seu processamento de latência extremamente baixa, o eXpressSFU consegue suportar um número substancialmente maior de usuários, mesmo com um único núcleo ARM, como mostra a Figura 14. A degradação de qualidade se torna perceptível acima de 300 usuários simultâneos, quando a latência total de processamento de pacotes excede 20,5 ms — consistente com o limite de latência discutido na §3. O impacto correspondente sobre o throughput é ilustrado na Figura 15, que compara o goodput obtido sob carga crescente. Enquanto o eXpressSFU consegue manter a taxa de bits ótima, o Jitsi Meet deixa de encaminhar vídeo por completo devido ao significativo relatório de descarte de quadros dos usuários.

Figura 14: Latência total de processamento de todos os usuários por pacote de mídia do eXpressSFU ao usar um único núcleo ARM. Quando a latência de processamento excede 20,5 ms, ocorre descarte de quadros no lado do receptor.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 15: Comparação de goodput. Usando um único núcleo, o eXpressSFU consegue suportar até 300 usuários a 1080p@24fps sem degradação de qualidade, alcançando goodput 26× melhor.
(imagem omitida nesta tradução; consulte o artigo original)

[1] O tempo por pacote para clonar um pacote aumenta com o número de usuários; acreditamos que isso ocorre porque todos os pacotes são clonados pelo único núcleo que processa aquele pacote antes de qualquer processamento adicional, em vez de se fazer melhor uso dos caches pequenos. Mesmo assim, o desempenho medido melhora.

7.4 Eficácia da adaptação de taxa de bits

O eXpressSFU não altera a adaptação de taxa de bits no controlador de SVC; o feedback do receptor é enviado ao controlador de taxa de bits no host, que então ajusta a seleção de fluxos na SmartNIC, conforme descrito na §5.1. Para demonstrar que o eXpressSFU tem impacto desprezível sobre o controle de taxa de bits, limitamos a largura de banda de download do receptor a várias taxas (por exemplo, 2 Mbps, 1 Mbps). Como mostra a Figura 16, a taxa de envio do eXpressSFU acompanha de perto a largura de banda disponível do receptor: entregando 1080p quando a largura de banda é irrestrita, reduzindo para 720p e depois 360p à medida que a largura de banda diminui, e restaurando a qualidade maior quando a largura de banda se recupera. Essa adaptação dinâmica preserva a QoE do usuário. Antes de cada troca de qualidade, o plano de controle no lado do host emite uma solicitação de quadro-chave (Full Intra Request) ao remetente — um evento que poderia ser facilmente interceptado na SmartNIC para uma coordenação host–SmartNIC ainda mais estreita.

Figura 16: Adaptação de taxa de bits SVC medida no eXpressSFU. O eXpressSFU consegue adaptar efetivamente a taxa de envio de acordo com a largura de banda variável do receptor (taxa de envio, taxa de recepção e largura de banda disponível ao receptor, com marcação das mudanças de largura de banda).
(imagem omitida nesta tradução; consulte o artigo original)

7.5 Consumo de energia

Para avaliar a eficiência operacional, medimos e comparamos o consumo total de energia do pacote entre o eXpressSFU e o Jitsi Meet usando um medidor de energia de tomada. Em nossa configuração, o Jitsi roda inteiramente na CPU do host, com o BlueField-2 operando como NIC[2] [41]. Por outro lado, o eXpressSFU aproveita tanto a CPU do host quanto os núcleos ARM, com o BlueField-2 operando em modo DPU [40]. Em estado ocioso, o servidor consome 113 W em ambos os casos; a Figura 17 reporta a potência adicional atribuível à computação. O consumo de energia do Jitsi aumenta acentuadamente com o número de usuários, chegando a mais de 55 W com 250 usuários. Em contraste, o eXpressSFU apresenta uma curva de crescimento muito mais plana, permanecendo abaixo de 22 W. Essa eficiência decorre do offload das funções críticas de latência do plano de dados para a SmartNIC, que aproveita a aceleração por lotes de pacotes executando em núcleos ARM energeticamente eficientes. Como resultado, o eXpressSFU reduz os custos operacionais e a emissão térmica, ao mesmo tempo em que sustenta uma capacidade de atendimento superior.

Figura 17: Comparação de potência computacional entre Jitsi e eXpressSFU. O eXpressSFU reduz o uso de energia em até 60%.
(imagem omitida nesta tradução; consulte o artigo original)

8 DISCUSSÃO

Limitação da SmartNIC atual. As operações criptográficas são uma fonte significativa de sobrecarga na SFU. Embora a SmartNIC Nvidia BlueField-2 suporte aceleração criptográfica por hardware à taxa de linha, ela se limita ao IPsec, em que as chaves de criptografia são negociadas de ponta a ponta entre as NICs — fora do controle de aplicações em espaço de usuário, como as construídas com DPDK. Além disso, as suítes criptográficas comumente usadas em videoconferência — AES--GCM-128 e AES-GCM-256 — não são suportadas pelo driver NVIDIA MLX5 Crypto na plataforma BlueField-2 [20].

Oportunidades com a nova plataforma SmartNIC. O NVIDIA BlueField-3 resolve essa restrição ao introduzir um segundo módulo de hardware criptográfico acessível a aplicações em espaço de usuário. Embora esse módulo ofereça alta vazão para criptografia e descriptografia de arquivos grandes, o projeto de sua API é inadequado para uma SFU (ver Apêndice §B). Ainda assim, o BlueField-3 melhora significativamente a capacidade computacional geral ao dobrar o número de núcleos ARM em relação ao BlueField-2. Além disso, como o plano de controle que roda na CPU do host permanece leve, implantar múltiplas SmartNICs em um único servidor poderia aumentar ainda mais a escalabilidade.

Escalonamento híbrido consciente de latência. Uma escolha de projeto alternativa é dedicar alguns núcleos da CPU do host ao plano de controle, alocando a maioria a um plano de dados baseado em DPDK, como na arquitetura eXpressSFU proposta. Embora essa configuração funcione bem para conferências pequenas (15–50 usuários) e de baixa qualidade (360p, 720p), ela enfrenta limites de escalabilidade em cenários grandes (100+ usuários) e de alta qualidade (1080p), devido à latência inerente de ida e volta do PCIe entre a CPU do host e o hardware da NIC [7, 23]. Assim, as conferências podem ser escalonadas de forma flexível em núcleos ARM ou de CPU, conforme seu orçamento específico de latência. O projeto detalhado de tal mecanismo de balanceamento de carga está além do escopo deste artigo e será explorado em trabalhos futuros.

9 TRABALHOS RELACIONADOS

Com o advento da videoconferência e as limitações da transmissão em tempo real baseada em MCU, desenvolveu-se a arquitetura SFU [26]. Diferentemente da MCU, que mixa os fluxos de mídia, uma SFU encaminha os fluxos de um participante aos outros sem processar o conteúdo, reduzindo a carga do servidor e transferindo a computação para os clientes. A evolução da SFU esteve intimamente alinhada ao sucesso e à ampla adoção do WebRTC. Numerosos frameworks foram desenvolvidos para suportar a arquitetura SFU, incluindo o Jitsi, um dos frameworks pioneiros de Videobridge com suporte a comunicação multiusuário, o MicroTalk SFU [17], o Kurento, que suporta tanto MCU quanto SFU [33], além do Licode [36] e do Janus [31]. Nos últimos anos, a SFU foi adaptada para aplicações emergentes como a conferência de realidade virtual baseada em metaverso [28]. Apesar dessas vantagens, as SFUs enfrentam desafios de escalabilidade, particularmente sob carga de pico e durante a replicação de pacotes para múltiplos usuários [45]. Trabalhos recentes trataram isso usando switches de rede P4 para descarregar operações da SFU, viabilizando duplicação e multicast de pacotes de baixo custo e eficientes [54].

10 CONCLUSÃO

Neste trabalho, apresentamos o eXpressSFU, uma re-arquitetura do sistema SFU convencional que aumenta significativamente a escalabilidade ao desacoplar de forma inteligente o pipeline de processamento de pacotes de mídia do caminho de controle

[2]No modo NIC, o BlueField-2 funciona como um adaptador ConnectX-6, com seus núcleos ARM contornados. Embora o Jitsi em si não precise dos núcleos ARM, uma NIC dedicada ainda é necessária para a transmissão e a recepção de pacotes, já que a interface integrada à placa não consegue fornecer largura de banda suficiente.

lento. Esse projeto viabiliza o processamento de mídia no caminho rápido diretamente na SmartNIC, preservando o controlador de taxa de bits existente no host, o que facilita a integração e a adoção sem atritos. O eXpressSFU também suporta tanto Simulcast quanto codificação de vídeo escalável (SVC) de forma transparente, garantindo compatibilidade com uma ampla gama de dispositivos com capacidades variadas.

Os resultados experimentais mostram que o eXpressSFU consegue acomodar até 3× mais usuários simultâneos no cenário mais desafiador — videoconferências de grande escala a 1080p@24fps — ao mesmo tempo em que reduz o consumo de energia computacional em até 60% em comparação com a linha de base.

AGRADECIMENTOS

Agradecemos sinceramente ao nosso shepherd, Daehyeok Kim, e aos revisores anônimos por seu valioso feedback. Agradecemos também a Evan Ram por suas contribuições ao desenvolvimento do eXpressSFU e a CJ Herman por sua ajuda com a configuração do testbed. Este trabalho foi apoiado pela National Science Foundation sob o Grant nº 1908910, pelo Institute of Information & Communications Technology Planning & Evaluation (IITP) sob os Grants nº RS-2024-00405128 e 2025-RS-2020--II201819, financiados pelo Governo da Coreia (MSIT), e pela National Research Foundation of Korea (NRF) sob o Grant nº RS-2025-24535401. Sangtae Ha é o autor correspondente.

REFERÊNCIAS (NO IDIOMA ORIGINAL)

[1] Dr. Bernard D. Aboba. Codec-Independent Selective Forwarding. https://datatracker.ietf.org/doc/draft-aboba-avtcore-sfu-rtp/00/. Accessed on January 4, 2025.
[2] Emmanuel André, Nicolas Le Breton, Augustin Lemesle, Ludovic Roux, and Alexandre Gouaillard. Comparative study of webrtc open source sfus for video conferencing. In 2018 Principles, Systems and Applications of IP Telecommunications (IPTComm), pages 1–8. IEEE, 2018.
[3] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman. The Secure Real-time Transport Protocol (SRTP). https://datatracker.ietf.org/doc/html/rfc3711, 2004.
[4] Edan Brooke. Setting up a self-hosted jitsi meet instance with multiple video bridges. https://shorturl.at/46LUN, 2023. Accessed: 2025-03-31.
[5] Ana Cárdenas, Mario García-Molina, Salvador Sales, and José Capmany. A new model of bandwidth growth estimation based on the gompertz curve: Application to optical access networks. Journal of Lightwave Technology, 22(11):2460, 2004.
[6] Gaetano Carlucci, Luca De Cicco, and Saverio Mascolo. Controlling queuing delays for real-time communication: the interplay of e2e and aqm algorithms. SIGCOMM Comput. Commun. Rev., 46(3), July 2018.
[7] Xuzheng Chen, Jie Zhang, Ting Fu, Yifan Shen, Shu Ma, Kun Qian, Lingjun Zhu, Chao Shi, Yin Zhang, Ming Liu, et al. Demystifying datapath accelerator enhanced off-path smartnic. In 2024 IEEE 32nd International Conference on Network Protocols (ICNP), pages 1–12. IEEE, 2024.
[8] Co, Industry Research. Video conferencing market: Future demand and top key players analysis: 2031, Nov 2023. Accessed on February 2, 2024.
[9] Contributors. Jicofo GitHub Repository. https://github.com/jitsi/jicofo, 2024. Accessed on February 2, 2024.
[10] Contributors. Jitsi GitHub Repository. https://github.com/jitsi, 2024. Accessed on February 2, 2024.
[11] Contributors. Jitsi Meet GitHub Repository. https://github.com/jitsi/jitsi-meet, 2024. Accessed on February 2, 2024.
[12] Contributors. Jitsi Videobridge GitHub Repository. https://github.com/jitsi/jitsi-videobridge, 2024. Accessed on February 2, 2024.
[13] Contributors. Mediasoup GitHub Repository. https://github.com/versatica/mediasoup, 2024. Accessed on February 2, 2024.
[14] David Curry. Zoom revenue and usage statistics (2025). https://www.businessofapps.com/data/zoom-statistics/, 2025. Accessed: 2025-04-09.
[15] Hans L. Cycon, Thomas C. Schmidt, Matthias Wahlisch, Detlev Marpe, and Martin Winken. A temporally scalable video codec and its applications to a video conferencing system with dynamic network adaption for mobiles. IEEE Transactions on Consumer Electronics, 57(3):1408–1415, 2011.
[16] Mallesham Dasari, Edward Lu, Michael W Farb, Nuno Pereira, Ivan Liang, and Anthony Rowe. Scaling vr video conferencing. In 2023 IEEE Conference Virtual Reality and 3D User Interfaces (VR), pages 648–657. IEEE, 2023.
[17] MiroTalk Developers. MiroTalk SFU: Secure, RealTime Video Conferencing. https://sfu.mirotalk.com/, 2025. Accessed: 2025-04-10.
[18] Tabin Dharanikota, Dennis Edens, Rajesh Abbi, and Sudheer Dharanikota. Forecasting bandwidth utilization growth. DTS whitepaper, Dec, 2018.
[19] MDN Web Docs. RTCPeerConnection.getStats() method. https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection/getStats, 2024. Accessed: 2025-10-21.
[20] DPDK. NVIDIA MLX5 Crypto Driver. https://doc.dpdk.org/guides/cryptodevs/mlx5.html. Accessed: 2025-04-25.
[21] DPDK. Struct Reference. https://doc.dpdk.org/api/structrte__crypto__sym__xform.html, 2025. Accessed: 2025-10-21.
[22] Alexandros Eleftheriadis, M Reha Civanlar, and Ofer Shapiro. Multipoint videoconferencing with scalable video coding. Journal of Zhejiang University-Science A, 7:696–705, 2006.
[23] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, Vivek Bhanu, Adrian Caulfield, Eric Chung, et al. Azure accelerated networking:{SmartNICs} in the public cloud. In 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), pages 51–66, 2018.
[24] National Center for Educational Statistics. How many students take distance learning courses at the postsecondary level? https://nces.ed.gov/fastfacts/display.asp?id=80&utm_source=chatgpt.com. Accessed on January 4, 2025.
[25] Google. VP9. https://developers.google.com/media/vp9. Accessed on January 4, 2025.
[26] Boris Grozev, Lyubomir Marinov, Varun Singh, and Emil Ivov. Last n: relevance-based selectivity for forwarding video in multimedia conferences. In Proceedings of the 25th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video, NOSSDAV '15, page 19–24, New York, NY, USA, 2015. Association for Computing Machinery.
[27] M. Handley, V. Jacobson, and C. Perkins. SDP: Session Description Protocol. https://datatracker.ietf.org/doc/html/rfc4566, 2006.
[28] Yong-Hao Hu, Kenichiro Ito, and Ayumi Igarashi. Improving real-time communication for educational metaverse by alternative webrtc sfu and delegating transmission of avatar transform. In 2023 International Conference on Consumer Electronics-Taiwan (ICCE-Taiwan), pages 201–202. IEEE, 2023.
[29] SNS Insider. Video Conferencing Market Set to Surge: Aiming for USD 25 Billion by 2032 Driven by Remote Work and Technological Advancements. https://tinyurl.com/26dxdexr. Accessed on January 4, 2025.
[30] Intel. Cryptodev Scheduler Poll Mode Driver Library. https://doc.dpdk.org/guides/cryptodevs/scheduler.html. Accessed on January 4, 2025.
[31] Janus. Janus: the general purpose webrtc server. https://github.com/meetecho/janus-gateway, n.d. Accessed: 2025-01-14.
[32] Stewart Jones, David Johnstone, and Roy Wilson. An empirical evaluation of the performance of binary classifiers in the prediction of credit ratings changes. Journal of Banking Finance, 56:72–85, 2015.
[33] Kurento. Kurento Media Server (KMS). https://doc-kurento.readthedocs.io/en/latest/index.html, n.d. Accessed: 2025-01-14.
[34] Jesús Leganés-Combarro. WebRTC Horizontal Scaling. https://piranna.github.io/2021/09/26/WebRTC-horizontal-scaling/, September 2021. Accessed: 2025-03-31.
[35] Xianshang Lin, Yunfei Ma, Junshao Zhang, Yao Cui, Jing Li, Shi Bai, Ziyue Zhang, Dennis Cai, Hongqiang Harry Liu, and Ming Zhang. Gso-simulcast: global stream orchestration in simulcast video conferencing systems. In Proceedings of the ACM SIGCOMM 2022 Conference, pages 826–839, 2022.
[36] Lynckia. Licode: Open source webrtc communications platform. https://lynckia.com/licode/, n.d. Accessed: 2025-01-14.
[37] R. Mahy, P. Matthews, and J. Rosenberg. Traversal Using Relays around NAT (TURN): Relay Extensions to STUN. https://datatracker.ietf.org/doc/html/rfc5766, 2010.
[38] NVIDIA. DOCA AES-GCM. https://docs.nvidia.com/doca/sdk/doca+aes-gcm/index.html. Accessed: 2025-04-25.
[39] NVIDIA. NVIDIA Bluefield-2 SmartNIC. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-2-dpu.pdf. Accessed on January 4, 2025.
[40] NVIDIA. NVIDIA Bluefield DPU mode. https://docs.nvidia.com/doca/archive/2-9-0-cx8/bluefield+modes+of+operation/index.html#src-3453016816_id-.BlueFieldModesofOperationv2.9.1-SmartNICmode. Accessed on Auguest 11, 2025.
[41] NVIDIA. NVIDIA Bluefield NIC mode. https://docs.nvidia.com/doca/archive/2-9-0-cx8/bluefield+modes+of+operation/index.html#src-3453016816_id-.BlueFieldModesofOperationv2.9.1-NICMode. Accessed on January 4, 2025.
[42] NVIDIA. NVIDIA DOCA Comm Channel. https://docs.nvidia.com/doca/archive/doca-v1.5.0/pdf/comm-channel-samples.pdf. Accessed on January 4, 2025.

[43] NVIDIA. Scalable Function. https://docs.nvidia.com/doca/archive/doca-v2.2.0/scalable-functions/index.html. Accessed on January 4, 2025.
[44] Stefano Petrangeli, Dries Pauwels, Jeroen van der Hooft, Tim Wauters, Filip De Turck, and Jürgen Slowack. Improving quality and scalability of webrtc video collaboration applications. In Proceedings of the 9th ACM Multimedia Systems Conference, MMSys '18, page 533–536, New York, NY, USA, 2018. Association for Computing Machinery.
[45] George Politis, Boris Grozev, Paweł Domas, Emil Ivov, and Thomas Noël. Experimental evaluation of dynamic switching between one-on-one and group video calling. In 2018 Principles, Systems and Applications of IP Telecommunications (IPTComm), pages 1–7, 2018.
[46] Miroslav Ponec, Sudipta Sengupta, Minghua Chen, Jin Li, and Philip A. Chou. Multi-rate peer-to-peer video conferencing: A distributed approach using scalable coding. In 2009 IEEE International Conference on Multimedia and Expo, pages 1406–1413, 2009.
[47] Linux Foundation Collaborative Projects. Data Plane Development Kit. https://www.dpdk.org/. Accessed on January 4, 2025.
[48] Linux Foundation Collaborative Projects. Open vSwitch. https://www.openvswitch.org/. Accessed on January 4, 2025.
[49] E. Rescorla and N. Modadugu. Datagram Transport Layer Security Version 1.2. https://datatracker.ietf.org/doc/html/rfc6347, 2012.
[50] J. Rosenberg. Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal for Offer/Answer Protocols. https://datatracker.ietf.org/doc/html/rfc5245, 2010.
[51] J. Rosenberg, R. Mahy, P. Matthews, and D. Wing. Session Traversal Utilities for NAT (STUN). https://datatracker.ietf.org/doc/html/rfc5389, 2008.
[52] Janto Skowronek, Alexander Raake, Gunilla H Berndtsson, Olli S Rummukainen, Paolino Usai, Simon NB Gunkel, Mathias Johanson, Emanuël AP Habets, Ludovic Malfait, David Lindero, et al. Quality of experience in telemeetings and videoconferencing: a comprehensive survey. IEEE Access, 10:63885–63931, 2022.
[53] Statista. Zoom daily meeting participants worldwide 2020. https://shorturl.at/YdwU3, 2021. Accessed: 2025-04-25.
[54] Pavlos Tsikrikas and George Xylomenos. A selective forwarding unit implementation in p4. In 2024 IEEE Conference on Standards for Communications and Networking (CSCN), pages 181–186, 2024.
[55] Justin Uberti, Stefan Holmer, Magnus Flodman, Danny Hong, and Jonathan Lennox. RTP Payload Format for VP9 Video. https://datatracker.ietf.org/doc/draft-ietf-payload-vp9/10/.
[56] www.statista.com. Use of collaboration tools in the united states 2023, by remote work policy. https://shorturl.at/ylyd4. Accessed on January 4, 2025.

APÊNDICES

A. Sobrecarga do PCIe sobre a pontualidade dos quadros

Para examinar como a latência do PCIe afeta a entrega de vídeo em tempo real, conduzimos um microexperimento que emula o comportamento de *fan-out* de pacotes da SFU. O tráfego de vídeo exibe um padrão periódico de rajadas, em que os pacotes chegam à SFU a cada intervalo de captura de vídeo — aproximadamente a cada 41 ms para um fluxo de 24 fps. No WebRTC, os receptores mantêm apenas um jitter buffer mínimo (tipicamente algumas dezenas de milissegundos) para manter baixa a latência de ponta a ponta. Assim, cada receptor precisa obter um quadro completo dentro do mesmo intervalo de 41 ms para cumprir o prazo de renderização. Para modelar isso, implementamos um simples par cliente–servidor de eco UDP: o cliente envia uma rajada de 64 pacotes, representando 64 fluxos de vídeo simultâneos, e o servidor replica cada pacote 100 vezes (6.400 no total) para simular o fan-out para múltiplos receptores.

O experimento foi conduzido com cliente e servidor rodando em duas máquinas separadas, conectadas por um switch L2 Top-of-Rack para minimizar a latência de rede. O processo do servidor foi executado em um único núcleo de CPU do host ou em um único núcleo ARM da SmartNIC. Foi imposto um prazo de ponta a ponta de 20 ms, assumindo que um quadro de vídeo é composto por dois pacotes. Ao rodar na CPU do host, apenas 8,4% do total de 25.600 requisições cumpriram o prazo do quadro. Em contraste, executar a mesma carga de trabalho no ARM da SmartNIC alcançou 57,1% de entrega no prazo. Esses resultados destacam que eliminar a travessia do PCIe entre o host e a NIC melhora significativamente a pontualidade dos quadros em cargas de trabalho de vídeo em tempo real.

B. Limitação do engine criptográfico na plataforma BlueField-3

Embora o offload de hardware na plataforma BlueField-3 alivie efetivamente os núcleos ARM de operações criptográficas computacionalmente intensivas, ele introduz latência não trivial devido ao seu projeto de API de memória compartilhada [38]. Cada requisição de criptografia ou descriptografia exige que o núcleo ARM copie o payload de entrada (DPDK mbuf) para uma região de memória compartilhada pré-alocada (buffers DOCA) acessível ao engine criptográfico, emita um comando para iniciar o processamento e faça polling ou espere a conclusão antes de copiar o resultado de volta. Essa movimentação de dados e sobrecarga de sincronização em várias etapas aumenta significativamente a latência de processamento de ponta a ponta — frequentemente dominando o tempo real de computação das operações AES-GCM. Como resultado, o mecanismo de offload criptográfico do BlueField-3, embora benéfico para cargas de trabalho orientadas a vazão, torna-se inadequado para pipelines de mídia críticos em latência, como a videoconferência em tempo real, em que até atrasos da ordem de microssegundos podem afetar a entrega de quadros e a experiência do usuário.